

Tidyverse

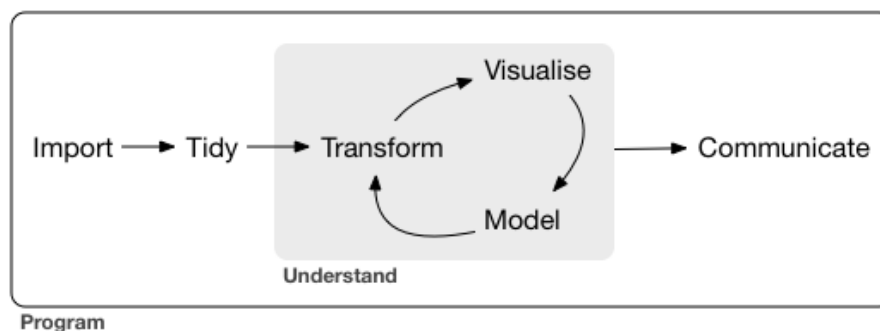
Data Analysis & Information Extraction

We have learn how to use R so far, and how to do some basic programming. Variables, data types, objects and key programming structures. In the next chapters of the course we are going to start to learn R as programming language for data analysis.

We will understand data science and analytics with R, focused in two aspects...

- Structured data (tables)
- Descriptive and diagnostic analysis (exploration and inference)

For doing that the learning structure of the next chapters is structured as follows:



- **Data Wrangling**

1. **Import:** Load data from different sources (databases, files, applications, etc.)
2. **Organize:** Save the data for the analysis.
3. **Transform:** Filter, create new variables, etc.

- **Data Exploration**

4. **Visualization:** Think questions, create new variables, etc.
5. **Exploratory Analysis:** Visualize and transform in a systematic way.

- **Modelling**

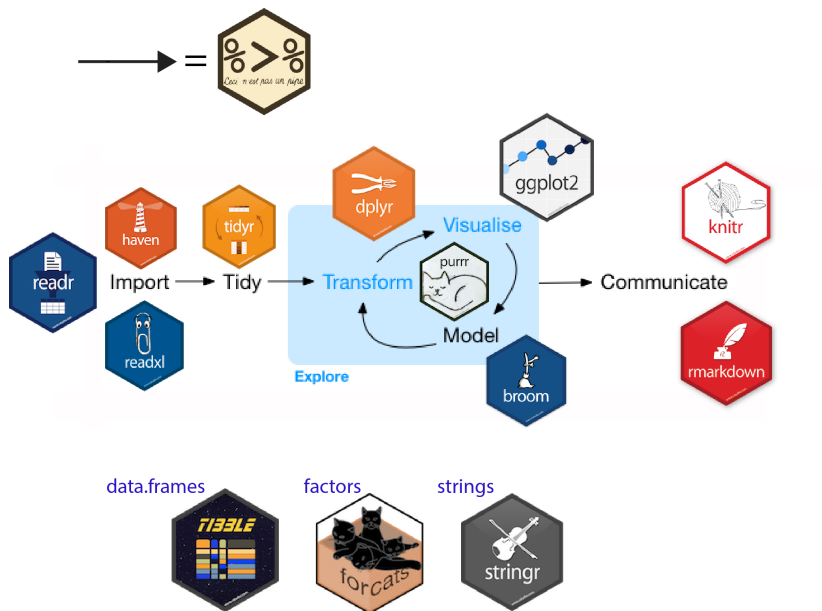
6. Check hypothesis and answer questions.

• Communication

7. Show the results.

The *tidyverse* ecosystem for data analysis

We want to analyze data in a **reproducible** way, and, for doing that we will use tools that can allow us develop strong analytics in R. Through the course we will use this set of packages that contains a full set of tools used for doing data science with R. Know more about [tidyverse packages](#)



So we need to start loading the **tidyverse** package (take into account that this package will load all tidyverse packages including tibble one). If you need to install tidyverse check the packages section on the first chapters of the book.

```
library(tidyverse)
```

Tibbles

All *tidyverse* packages use an special type of data structure called **tibble**, that is the next evolution of the data frames at R.

Tibbles are similar to data frames but they have special features that make them more interesting for data analysis.

Remember!! First all we need to load *tidyverse* package (take into account that this package will load all tidyverse packages including tibble one).

```
library(tidyverse)
```

Data Frames vs Tibbles

Printing

This is how iris data set looks as a data frame object:

```
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Now we are going to convert iris to a tibble, we will use tidyverse function `as_tibble()`:

```
iris_tb <- as_tibble(iris)
head(iris_tb)
```

```
# A tibble: 6 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
    <dbl>         <dbl>         <dbl>         <dbl> <fct>
1     5.1         3.5         1.4         0.2 setosa
2     4.9         3.0         1.4         0.2 setosa
3     4.7         3.2         1.3         0.2 setosa
4     4.6         3.1         1.5         0.2 setosa
```

Data Frames vs Tibbles

5	5	3.6	1.4	0.2 setosa
6	5.4	3.9	1.7	0.4 setosa

We see that in the console... * output of the data frame we only get the data displayed directly and unorganized. * output of the tibble shows a summary of dimensions as well as a compact visualization of the columns and types of each column.

Column Names

On tibbles column names don't need to be consecutive you can have names with spaces, like this example:

```
dt <- tibble(
  p = c("A", "B", "C", "D", "E"),
  x = 1:5,
  y = 2,
  `sum of values` = x + y # we can reference other columns
)
dt
```

```
# A tibble: 5 x 4
  p         x     y `sum of values`
<chr> <int> <dbl>         <dbl>
1 A         1     2             3
2 B         2     2             4
3 C         3     2             5
4 D         4     2             6
5 E         5     2             7
```

Selection

When selecting and modifying values you can work the same as a data frame but consider that the output of any selection will return a data frame always...

```
# first value of first column as DF
dt[1,1]
```

```
# A tibble: 1 x 1
  p
<chr>
1 A
```

Creating a tibble

If we want to manipulate the value we will need to do a second selection

```
# first value of first column
dt[1,1][[1]]
```

```
[1] "A"
```

You need to use name selection or double selection to access the values itself.

```
# first column as vector
dt$p
```

```
[1] "A" "B" "C" "D" "E"
```

```
# first column value as value
dt[[1]][1]
```

```
[1] "A"
```

Creating a tibble

In case you need to create a tibble manually, you can defined it as the following example.

```
dt <- tibble(
  p = c("A", "B", "C", "D", "E"),
  x = 1:5,
  y = 2,
  `sum of values` = x + y
)
dt
```

```
# A tibble: 5 x 4
  p         x     y `sum of values`
<chr> <int> <dbl>         <dbl>
1 A         1     2             3
2 B         2     2             4
3 C         3     2             5
4 D         4     2             6
5 E         5     2             7
```